resumer

DevOps et Cloud Computing

USTHB · Ingénieur 5 · Software Engineering · S9

Définitions · Comparaisons · Méthodes · Révision active

Synthèse des trois cours, des quatre énoncés pratiques et du TD1 disponibles au 11 novembre 2026. Enseignante : Ameur Imane. Le contenu porte surtout sur virtualisation, cloud et infrastructure ; aucun chapitre autonome sur CI/CD n'est présent dans les supports fournis.

Mise à jour : 11 novembre 2026

Sommaire

Chapitre 1 — Principes de virtualisation . . . 3

Chapitre 1 — Conteneurs, domaines et performance . . . 3

Chapitre 2 — Systèmes distribués et cloud . . . 4

Chapitre 2 — Déploiement et modèles de service . . . 5

Chapitre 2 — Edge, fog et infrastructure physique . . . 6

Chapitre 3 — Architecture du cloud . . . 7

TP1 et TP2 — Ce qu'il faut savoir expliquer . . . 8

TP3 et TP4 — Codespaces et OpenStack . . . 9

TD1 — Architecture, stockage et services . . . 10

Révision active et sources . . . 11

Chapitre 1 — Principes de virtualisation

Définition et vocabulaire

La **virtualisation** représente une ressource informatique sous une forme logique, découplée de certains détails physiques. Un même hôte peut ainsi exécuter plusieurs machines virtuelles isolées, chacune avec son système invité.

Terme	Signification
Hôte, host	Machine qui fournit les ressources physiques
Invité, guest	Système exécuté dans une VM
Hyperviseur	Couche qui crée, exécute et gère les VM et leurs ressources
vCPU	Ressource de processeur présentée à l'invité
Disque virtuel	Support logique de stockage de l'invité
Snapshot	État conservé à un instant, utile pour un retour en arrière

Chaîne conceptuelle : **matériel → hyperviseur → matériel virtuel → OS invité → applications**. L'abstraction ne permet pas pour autant d'exécuter sans contrainte n'importe quel OS sur n'importe quelle architecture.

Objectifs et limites

La consolidation augmente l'utilisation de serveurs auparavant peu chargés. Elle peut réduire matériel, énergie et espace, et facilite création d'environnements, tests, migration et reprise. Les gains chiffrés d'une présentation sont des illustrations, pas une garantie pour tout déploiement.

L'isolation réduit les interférences entre invités, mais plusieurs VM restent dépendantes d'un même hôte. Sa panne peut toucher toutes ses VM ; il faut une architecture adaptée pour la continuité de service. Un snapshot n'est pas une sauvegarde indépendante complète.

Types d'hyperviseurs

Critère	Type 1, bare metal	Type 2, hosted
Position	Directement au niveau matériel / noyau de virtualisation	Application d'un OS hôte
Usage classique	Infrastructure de production et centres de données	Poste personnel, développement et laboratoire
Exemples du cours	ESXi, Hyper-V, Xen, KVM	VirtualBox, VMware Workstation
Point de vigilance	Administration et capacité de l'hôte	Couche hôte supplémentaire et concurrence avec ses applications

Le classement décrit une architecture. Il ne faut pas conclure qu'un type 1 sera plus rapide pour toute charge dans toute configuration. Les détails d'intégration varient selon le produit.

Chapitre 1 — Conteneurs, domaines et performance

VM et conteneurs

Aspect	Machine virtuelle	Conteneur
Unité isolée	Matériel virtuel avec OS invité	Processus et environnement logiciel
Noyau	Propre à l'invité	Partagé avec l'hôte d'exécution
Poids et démarrage	Généralement plus importants	Généralement plus légers et rapides
Usage	OS différents, isolation de machines	Déploiement d'applications et dépendances

Un conteneur n'est pas simplement une petite VM avec son propre noyau. Une VM peut aussi héberger un moteur de conteneurs. Le choix suit les contraintes de compatibilité, d'isolation, d'exploitation et de densité.

Domaines de virtualisation

• **Serveur** : plusieurs rôles logiques partagent un matériel.

• **Bureau** : environnement de travail accessible à distance et administré centralement.

• **Réseau** : commutateurs, routeurs ou fonctions réseau logiques.

• **Stockage** : volumes logiques construits au-dessus de supports physiques.

• **Application** : environnement isolé et gestion des dépendances.

Surcoûts et optimisation

Le surcoût vient notamment de l'ordonnancement des vCPU, des E/S, de la gestion mémoire et des changements de contexte. Il faut mesurer la charge réelle plutôt que multiplier arbitrairement les ressources virtuelles.

Technique citée	But
Assistance matérielle VT-x / AMD-V	Faciliter l'exécution virtualisée
Pilotes paravirtualisés	Réduire le coût des E/S virtuelles
Ballooning	Ajuster la mémoire récupérable des invités
CPU pinning	Contrôler l'affectation aux cœurs
SR-IOV	Fournir un accès optimisé à certaines fonctions matérielles réseau

La chronologie utile est mainframes → virtualisation x86 → plateformes d'entreprise et cloud → conteneurs. Les dates et conditions commerciales parfois contradictoires du support ne sont pas utilisées comme faits à apprendre.

Chapitre 2 — Systèmes distribués et cloud

Le cloud s'appuie sur la distribution

Un système distribué réunit des machines indépendantes qui coordonnent leurs actions par messages. On recherche partage, extension de capacité, proximité géographique et continuité de service. La distribution crée aussi des difficultés de coordination, cohérence des répliques et pannes partielles.

Architecture	Principe	Limite à comprendre
Client-serveur	Clients consommateurs, serveur fournisseur	Un serveur unique peut devenir un point de panne ou un goulot

Architecture	Principe	Limite à comprendre
Pair-à-pair	Un nœud peut fournir et consommer	Coordination et découverte plus complexes
SOA	Services exposés via des interfaces et protocoles	Contrats et dépendances doivent rester maîtrisés

La SOA recherche couplage faible, réutilisation et interopérabilité. Dans le schéma du cours : le fournisseur publie, un registre permet la découverte, puis le consommateur invoque le service. Un objet expose des méthodes ; un composant regroupe des fonctions réutilisables ; un service expose une capacité accessible en réseau.

Définition et cinq caractéristiques

Le **cloud computing** donne un accès réseau à un ensemble de ressources configurables, provisionnées et libérées à la demande avec un effort de gestion limité.

Caractéristique	Sens	Exemple pédagogique
Libre-service à la demande	Le client demande ses ressources par console ou API	Créer une VM
Accès réseau étendu	Accès par mécanismes réseau usuels	Utiliser HTTPS ou SSH
Mutualisation	Ressources physiques partagées entre clients	Infrastructure multi-tenant
Élasticité rapide	Adapter les ressources à la variation de charge	Ajouter puis retirer des instances
Service mesuré	Quantifier l'utilisation	Heures de VM, stockage ou trafic

La **scalabilité** concerne la capacité à supporter une charge croissante ; l'**élasticité** adapte la capacité aux variations, dans les deux sens. L'automatisation nécessite une configuration : créer une VM ne prouve pas qu'un autoscaling est actif.

Chapitre 2 — Déploiement et modèles de service

Public, privé, hybride et communautaire

Modèle	Organisation	Compromis principal
Public	Services d'un fournisseur accessibles à une clientèle large	Facilité de provisionnement, dépendance au fournisseur
Privé	Usage exclusif d'une organisation	Contrôle, mais exploitation et capacité à financer
Hybride	Association de clouds distincts reliés pour des usages coordonnés	Flexibilité avec complexité d'intégration
Communautaire	Plusieurs organisations ayant des exigences communes	Partage des coûts et gouvernance commune

Une infrastructure privée n'est pas automatiquement plus sûre ; l'exploitation, les identités et les configurations comptent. Un cloud n'est pas simplement un serveur distant : les caractéristiques de service doivent être considérées.

Répartition des responsabilités

Modèle	Le client gère surtout	Le fournisseur prend en charge surtout
Sur site	Toute la pile	Pas de délégation cloud par défaut
IaaS	OS invité, mises à jour, applications, données et règles d'accès	Matériel, infrastructure physique et couche de virtualisation
PaaS	Code, données et configuration applicative	Infrastructure, OS et environnement de plateforme selon l'offre
SaaS	Usage, comptes, configuration et gouvernance des données	Application et pile de service selon le contrat

Exemples du cours : VM cloud pour IaaS, plateforme d'hébergement applicatif pour PaaS, application collaborative en ligne pour SaaS. La délégation ne supprime pas toutes les responsabilités du client. La sauvegarde, l'accès et la récupération doivent être prévus explicitement.

Les avantages attendus sont rapidité de mise à disposition, souplesse et investissement initial réduit. Les contraintes incluent coûts récurrents, dépendance réseau, sécurité, localisation des données et difficulté de migration. Les prix et les offres gratuites doivent être vérifiés au moment d'un déploiement ; aucun tarif n'est supposé ici.

Chapitre 2 — Edge, fog et infrastructure physique

Où traiter les données ?

Niveau	Emplacement	Utilité
Edge	Objet ou ressource proche de la source	Réponse locale, réduction des transmissions
Fog	Nœuds intermédiaires et passerelles	Agrégation et coordination de proximité
Cloud	Infrastructure distante mutualisée	Stockage massif, analyses et vision globale

Exemple : une caméra filtre localement, une passerelle agrège plusieurs flux, le cloud conserve des historiques. Les délais réels dépendent du réseau et de l'implémentation ; une appellation ne suffit pas à garantir une latence.

Du serveur au centre de données

Un serveur est aussi un rôle de service, pas seulement un format de boîtier. Les tours conviennent à de petits environnements ; les serveurs rack favorisent la densité ; les blades partagent un châssis et certaines ressources.

Un rack standard se décrit notamment en unités U : **1 U = 1,75 pouce = 44,45 mm** de hauteur. Le châssis accueille des modules ; le cluster associe des nœuds ; le centre de données rassemble calcul, stockage, réseau et infrastructure d'exploitation.

Infrastructure	Rôle
Réseau	Commutateurs, routeurs, pare-feu et interconnexion
Refroidissement	Maîtriser la température et les flux d'air
UPS	Maintenir temporairement l'alimentation lors d'une coupure

Infrastructure	Rôle
PDU	Distribuer l'énergie aux équipements
Exploitation	Surveillance, maintenance, remplacement et gestion des incidents

Des centres géographiquement répartis peuvent rapprocher les services des utilisateurs et limiter certains risques de panne. Cette répartition nécessite une architecture de données et de reprise appropriée.

Le plan du chapitre annonce des thèmes supplémentaires sur adoption, SLA et multi-cloud ; le PDF présent développe surtout les sections résumées ci-dessus. Ces thèmes annoncés ne sont pas traités ici comme des chapitres détaillés déjà disponibles.

Chapitre 3 — Architecture du cloud

Front end et back end

Partie	Contenu	Rôle
Front end	Terminal client, navigateur ou application, interface (portail, SaaS)	Permettre à l'utilisateur d'accéder aux services
Back end	Infrastructure physique, hyperviseurs (VMware, KVM), plateforme de gestion (OpenStack), stockage, sécurité et sauvegarde	Gérer les ressources, exécuter la logique métier, protéger les données

L'architecture vise des solutions évolutives, fiables, sécurisées et économiques : redondance et tolérance aux pannes pour la disponibilité, réduction des dépenses d'investissement en matériel.

Quatre briques d'architecture

Brique	Principe	Exemple cloud
N-Tiers	Couches présentation, logique métier, données ; séparation des responsabilités	Front sur S3/CloudFront, logique sur Lambda/EC2, données sur RDS/BigQuery
Machines virtuelles	Émulation logicielle d'un ordinateur : OS hôte, hyperviseur, OS invité	Isolation, portabilité, mutualisation
Virtualisation de fichiers	Découpler l'accès aux fichiers du stockage physique ; vue unifiée (VFS)	Dropbox, Google Drive, S3
SOA	Services indépendants, faiblement couplés, réutilisables, communiquant par API	Utilisateur, produit, commande, notification d'un site e-commerce

Chaque niveau N-Tiers peut évoluer indépendamment, ce qui simplifie les mises à jour et isole les pannes.

SOA : caractéristiques et protocoles

• **Couplage faible** : un service ne dépend pas de l'implémentation des autres.

• **Communication par API** : REST, gRPC, GraphQL, SOAP.

• **Réutilisabilité** et **indépendance technologique** : chaque service peut utiliser son propre langage.

• Avantages annoncés : déploiement rapide, mise à l'échelle ciblée, résilience, mise à jour d'un seul service.

Serverless et FaaS

Le développeur écrit des **fonctions** exécutées seulement en réponse à un événement (dépôt de fichier, modification de base, requête HTTP, message en file). Le fournisseur crée, exécute, met à l'échelle et arrête les ressources. Facturation **pay-as-you-go** : seule la durée d'exécution du code est payée, contre la durée de fonctionnement d'un serveur classique.

Fournisseur	Service
AWS	Lambda
Azure	Azure Functions
Google Cloud	Cloud Functions / Cloud Run
Open source	OpenFaaS, OpenWhisk, Knative

Limite à connaître (TD1) : la latence de démarrage à froid ; la concurrence provisionnée garde des environnements prêts.

TP1 et TP2 — Ce qu'il faut savoir expliquer

TP1 : VM locale, réseau, SSH et snapshot

Le laboratoire demande deux VM Linux, l'allocation de vCPU/RAM/disque, une duplication avec identités distinctes, des communications entre hôte et invités, puis un retour à un snapshot. Le nom affiché par l'hyperviseur peut différer du hostname interne.

Mode réseau	Communication typique	Point à retenir
NAT	Sortie via traduction d'adresses	Une entrée depuis l'extérieur demande généralement une redirection
Bridged	VM participante du réseau local	Adressage et filtrage du LAN s'appliquent
Host-only	Hôte et VM sur un réseau privé	Pas d'accès Internet par défaut
Réseau interne	VM entre elles	Hôte non connecté directement par défaut

Ces descriptions ne constituent pas une garantie de sécurité ; les règles de filtrage et la configuration réelle restent à vérifier.

```
hostname
whoami
ip a
sudo systemctl status ssh
ssh utilisateur@ADRESSE_VM
```

Les trois premières commandes permettent d'identifier la machine distante. `ssh localhost` teste le serveur local, sans prouver que le réseau entre machines fonctionne. Après restauration du snapshot, constater l'état réel du système ; arrêter un service n'est pas la même chose que restaurer un état antérieur.

TP2 : VM IaaS et serveur web

Chaîne à comprendre : choisir image/région/taille → créer la VM → restreindre les accès réseau → se connecter en SSH → installer Nginx → vérifier le service et l'accès HTTP → contrôler les ressources restantes.

```
sudo apt update
sudo apt install -y nginx
sudo systemctl enable --now nginx
```

```
sudo systemctl status nginx --no-pager
curl -I http://localhost
```

Un service actif localement peut rester inaccessible depuis un navigateur à cause du pare-feu, du réseau ou d'une mauvaise adresse. Le TP demande SSH depuis l'IP autorisée et HTTP selon la règle de l'enseignante. Arrêter une VM peut laisser des disques, adresses ou snapshots facturables : vérifier les ressources effectivement créées.

Cette section résume les énoncés ; elle ne prétend pas qu'un nouveau déploiement cloud a été effectué pour rédiger cette fiche.

TP3 et TP4 — Codespaces et OpenStack

TP3 : environnement de développement GitHub Codespaces

Le sujet porte le titre « TP2 » mais est distribué comme TP3. Chaîne : créer un dépôt → ouvrir un Codespace 2 cœurs → explorer Linux (`whoami`, `hostname`, `uname -a`, `df -h`, `free -h`, `uptime`) → installer et lancer Nginx → exposer le port 80 par redirection de port → observer l'usage → arrêter ou supprimer le Codespace.

```
sudo apt-get update
sudo apt-get install -y nginx
sudo nginx
curl -I http://localhost
```

Caractéristique cloud	Observation dans Codespaces
Libre-service à la demande	Création du Codespace sans intervention humaine
Accès réseau large	Éditeur et page Nginx accessibles depuis un navigateur
Mutualisation	Matériel, hyperviseur et réseau gérés par GitHub
Provisionnement rapide	Environnement prêt en quelques minutes ; type de machine modifiable
Service mesuré	Page de facturation : heures-cœur et Go-mois de stockage

Le port redirigé reste **privé** sauf demande explicite. Un Codespace arrêté ne consomme plus de calcul, mais son stockage reste compté : supprimer ceux qui ne servent plus.

TP4 : installation d'OpenStack avec DevStack

Sur Ubuntu Server 24.04 dédiée (recommandation de classe : 4 vCPU, 16 Go RAM, 80 Go disque, virtualisation imbriquée si possible) : mettre à jour le système → créer l'utilisateur `stack` avec sudo sans mot de passe → cloner DevStack → écrire `local.conf` → lancer `./stack.sh` en tant que `stack`, jamais avec sudo.

```
source openrc admin admin
openstack service list
openstack image list
openstack network list
```

Composant	Rôle
Keystone	Identité et authentification
Nova	Instances de calcul
Neutron	Réseaux, sous-réseaux, routage, groupes de sécurité
Glance	Images de VM

Composant	Rôle
Cinder	Volumes bloc
Horizon	Tableau de bord web (`http://<IP>/dashboard`)
Placement	Inventaire et allocation des ressources de calcul

DevStack sert au développement et à l'enseignement, pas à la production : il modifie la configuration système et réseau.

TD1 — Architecture, stockage et services

Plateforme de commerce en ligne

Une architecture en couches sépare interface utilisateur, traitement métier et persistance. Le **back end** comprend les services côté serveur ; la logique métier et l'accès aux données en font partie, ce ne sont pas nécessairement quatre serveurs distincts.

Couche / ressource	Proposition pour le cas du TD	Justification
Front end	Interface web	Interaction avec les utilisateurs
Logique métier	Commandes et paiements	Règles et traitements applicatifs
Données structurées	Base relationnelle, telle que RDS dans l'exemple AWS	Relations, intégrité et transactions
Images	Stockage objet, tel que S3 dans l'exemple	Stockage et distribution de fichiers avec métadonnées

Le besoin de 50 000 utilisateurs simultanés exige un dimensionnement et des mesures ; ce nombre seul ne permet pas de déduire une taille de VM. La disponibilité nécessite plusieurs instances, un routage vers les instances saines et une stratégie de reprise des données. Une sauvegarde permet une restauration ; elle ne remplace pas un basculement automatique.

Les microservices permettent de faire évoluer certains composants séparément, mais ajoutent échanges réseau et exploitation distribuée. Leur choix doit être justifié par les besoins et l'équipe ; plusieurs fonctions métier ne rendent pas un monolithe automatiquement inadapté.

Ressources virtuelles et contention

Pour trois départements, une VM par département fournit une séparation logique sur le serveur physique. L'hyperviseur attribue et ordonnance les ressources. Un invité trop consommateur peut dégrader les autres : prévoir limites, réservations et surveillance adaptées. La panne du seul hôte reste un risque commun.

Migration : déplacer une VM vers un autre hôte. **Réplication** : maintenir une copie selon la stratégie prévue, utilisable notamment pour une reprise. Une copie ne reste pas synchronisée spontanément ; il faut un mécanisme de réplication.

Stockage fichier, objet, bloc et VFS

Notion	Fonction
Partage de fichiers	Accès à une arborescence avec un protocole et des droits
SMB / NFS	Protocoles de partage de fichiers, fréquemment associés à Windows / Unix mais pas exclusifs à ces OS

Notion	Fonction
Stockage objet	Accès à des objets identifiés, avec données et métadonnées
Stockage bloc	Volumes vus comme blocs par un système, qui peut y créer un système de fichiers
VFS	Abstraction d'accès aux systèmes de fichiers au niveau du système d'exploitation

Pour des postes Windows et Linux, vérifier le protocole, les clients, les identités et les permissions réellement compatibles. Dire seulement « virtualisation de fichiers » ne résout pas ces exigences. Un VFS n'est pas une garantie automatique de chemins réseau identiques après déplacement des fichiers.

Le stockage objet convient au scénario d'images proposé ; le choix final dépend aussi des applications et contraintes d'accès. Pour protéger les données, distinguer **accès non autorisé** (identités, autorisations, chiffrement) et **perte de données** (sauvegardes, versions, réplication et essais de restauration). Un pare-feu ou le chiffrement ne suffit pas à restaurer des données effacées.

FaaS et traitement d'images

Le traitement événementiel peut suivre : dépôt de l'image dans un stockage → notification de création d'objet → fonction de traitement → résultat stocké. Le déclencheur cloud est l'événement configuré après le dépôt, pas le simple clic sur un bouton de l'interface.

Le modèle FaaS évite de gérer directement les serveurs d'exécution. Le coût dépend de la tarification du service, notamment invocations, durée et ressources, avec d'autres coûts possibles ; il n'est pas limité universellement au seul temps d'exécution. Le cold start peut ajouter de la latence ; une capacité préinitialisée, quand le service la propose, constitue un compromis de coût et de délai. Le TD ne fournit pas de tarifs à mémoriser.

SOA, SOAP et REST

Des services Java, Python et Node.js peuvent coopérer grâce à des contrats d'interface et des formats/protocoles communs. Le couplage faible limite certaines dépendances ; il ne signifie pas absence de dépendances.

SOAP est un protocole de messages utilisant XML. **REST** est un style d'architecture fondé notamment sur des ressources et des interactions définies ; REST n'impose pas JSON, même si ce format est courant dans les exemples web.

Une panne de notification ne doit pas nécessairement arrêter un paiement, mais cela dépend de l'implémentation. Une notification asynchrone avec reprise et gestion des erreurs limite la propagation ; un appel synchrone mal géré peut au contraire bloquer le flux. L'indépendance annoncée dans un schéma ne suffit pas à garantir la résilience.

Révision active et sources

1. **Une VM suffit-elle à faire un cloud ?** Non, considérer aussi les caractéristiques de service.

2. **Quel modèle laisse l'OS invité au client ?** IaaS.

3. **Quel mode pour une VM directement adressable sur le LAN ?** Bridged, sous réserve du réseau et du filtrage.

4. **Pourquoi distinguer snapshot et sauvegarde ?** Un état de retour local ne couvre pas tous les scénarios de perte de l'hôte ou du stockage.

5. **Pourquoi comparer curl local et accès navigateur distant ?** Ils ne testent pas le même chemin réseau.

6. **Que signifie élasticité dans les deux sens ?** Ajouter des ressources puis les libérer lorsque la demande baisse.

7. **Une sauvegarde garantit-elle la continuité immédiate après une panne ?** Non : elle doit être distinguée de la redondance et du basculement.

8. **REST implique-t-il JSON ?** Non, c'est un style architectural ; plusieurs représentations sont possibles.

9. **Un service de notification en panne bloque-t-il toujours le paiement ?** Cela dépend des dépendances et de la gestion des erreurs.

Sources locales : `COURS/Chapter1-Virtualization.pdf` (49 pages), `COURS/Chapter2-Cloud-Computing.pdf` (117 pages), `COURS/Chapter3-Cloud-Architecture.pdf` (39 pages), `TP/TP1.pdf` (6 pages), `TP/TP1-support-network-modes.pdf` (21 pages), `TP/TP2.pdf` (4 pages), `TP/TP3.pdf` (4 pages), `TP/TP4.pdf` (6 pages), `TD/TD1-sol.pdf` (3 pages, cinq exercices corrigés). Les notes du TD précisent les réponses trop générales sur disponibilité, stockage et indépendance des services. Les exemples de fournisseurs, dates historiques et chiffres commerciaux non nécessaires à la compréhension ont été écartés.